Resolución Reto Técnico — Ingeniero Cloud

Autor: Angel Ortega | Fecha: Octubre 2026

Parte I: Preguntas Teóricas

1. ¿Cuál es la diferencia entre nube pública, privada e híbrida?

Más allá de quién es el dueño del hardware, la diferencia real en el día a día pasa por cómo se administra el presupuesto (CAPEX vs. OPEX), qué tanto control tenemos sobre la capa física y hasta dónde nos permiten llegar las regulaciones de datos.

- **Nube Pública (ej. Azure, AWS, GCP):** El proveedor administra todo el centro de datos físico, redes base y virtualización, y nosotros consumimos recursos bajo demanda pagando por uso (OPEX). La infraestructura física es compartida con otros clientes (multi-tenant), pero está aislada lógicamente a nivel de red e hipervisor. Su mayor ventaja es poder escalar servicios en minutos sin tener que comprar ni mantener servidores físicos.
- **Nube Privada (ej. entornos sobre VMware, Proxmox, OpenStack o Azure Stack):** Es infraestructura dedicada únicamente a una organización, ya sea en un data center propio o alojada con un tercero. Aquí tenemos control total del hardware, del almacenamiento y de las políticas de red físicas. El costo inicial y operativo suele ser más alto (CAPEX y mantenimiento continuo), pero se sigue utilizando cuando existen normativas estrictas de soberanía de datos o aplicaciones legacy que no pueden salir a una nube pública.
- **Nube Híbrida:** Combina la infraestructura privada (on-premise) con uno o más proveedores de nube pública mediante enlaces de red seguros (VPN Site-to-Site o enlaces dedicados como ExpressRoute / Direct Connect). En el sector financiero es el modelo más común: permite mantener la base de datos transaccional o el core en el centro de datos local por temas regulatorios y de latencia, mientras los canales digitales (web, banca móvil, APIs) corren y escalan en la nube pública según la demanda de los usuarios.

Criterio	Nube Pública	Nube Privada	Nube Híbrida
Modelo de Costos	OPEX (Pago por consumo)	CAPEX + Costo de operación	Mixto (CAPEX local + OPEX cloud)
Tenencia / Hardware	Multi-tenant (Aislamiento lógico)	Single-tenant (Dedicado)	Combinado según la carga de trabajo
Escalabilidad	Elástica e inmediata	Limitada al hardware instalado	Alta (permite desborde hacia la nube)

Caso típico de uso	Apps nativas, microservicios, frontends	Core transaccional, datos regulados	Banca digital conectada a sistemas base

2. Describe tres prácticas de seguridad en la nube.

En la nube trabajamos bajo el modelo de responsabilidad compartida: el proveedor asegura el centro de datos, pero asegurar lo que configuramos adentro es tarea nuestra:

1. **Identidad como perímetro y Principio de Mínimo Privilegio (IAM / RBAC):** Nadie —ni usuarios ni servicios— debe tener más permisos de los que necesita para su tarea puntual. Para personas, esto implica usar autenticación multifactor (MFA), roles granulares (RBAC) y accesos temporales para tareas de administración. Para las aplicaciones, la regla principal es no quemar credenciales ni cadenas de conexión en el código o en variables de texto plano; en su lugar, se usan identidades administradas (Managed Identities en Azure o IAM Roles en AWS) para que el backend se autentique directo contra la base de datos o el gestor de secretos.
2. **Segmentación de Red y Tráfico Privado (Defensa en Profundidad):** El hecho de estar en la nube no significa que los recursos deban tener IPs públicas. Solo el punto de entrada (como un balanceador con WAF) da la cara a internet. El backend y las bases de datos se despliegan en subredes privadas separadas, restringiendo el paso entre capas con grupos de seguridad (NSGs / Security Groups). Además, para conectar servicios PaaS (bases de datos o cuentas de almacenamiento), se utilizan enlaces privados (Private Endpoints / Private Link) para que el tráfico nunca salga de la red interna del proveedor.
3. **Cifrado de Datos y Gestión Centralizada de Secretos:** Toda información sensible debe viajar cifrada en tránsito (forzando TLS 1.2 o 1.3 de extremo a extremo) y guardarse cifrada en reposo en discos, bases de datos y almacenamiento de objetos. Las llaves criptográficas, certificados y tokens de terceros se centralizan en un servicio dedicado (como Azure Key Vault o AWS Secrets Manager/KMS) con rotación periódica y auditoría de accesos, evitando que cualquier desarrollador o administrador tenga acceso directo a las claves de producción.

3. ¿Qué es la IaC, cuáles son sus principales beneficios y 2 herramientas principales?

Infraestructura como Código (IaC) consiste en definir y desplegar la infraestructura (redes, servidores, bases de datos, reglas de firewall) mediante archivos de configuración declarativos en lugar de crearlos a mano haciendo clic en el portal web del proveedor.

Beneficios principales en operación:

- **Estandarización y cero 'Configuration Drift':** Evita el clásico problema de que el ambiente de desarrollo funcione distinto al de producción porque alguien cambió un parámetro a mano y no lo documentó.

- **Control de versiones y trazabilidad:** Al vivir en un repositorio Git, cada cambio en la infraestructura pasa por un Pull Request. Queda registro exacto de quién abrió un puerto o modificó un recurso, cuándo lo hizo y por qué.
- **Rapidez y recuperación ante fallos:** Permite levantar entornos completos en minutos o recrear toda la arquitectura en otra región si ocurre una caída grave.

Dos herramientas de IaC y sus características:

1. **Terraform (HashiCorp):** Es una herramienta agnóstica al proveedor que utiliza el lenguaje declarativo HCL. Su característica más fuerte es que permite manejar recursos de Azure, AWS, GCP o Kubernetes con la misma sintaxis y flujo de trabajo. Funciona manteniendo un archivo de estado (tfstate), normalmente guardado en un almacenamiento remoto con bloqueo, lo que le permite comparar qué existe realmente en la nube contra lo que está escrito en el código antes de aplicar cualquier cambio.
2. **Azure Bicep / ARM Templates (o AWS CloudFormation en AWS):** Son las herramientas nativas de cada nube (Bicep compila sobre ARM en Azure). Su gran ventaja frente a herramientas externas es que no necesitas administrar ni asegurar un archivo de estado (.tfstate), ya que el propio plano de control de la nube lleva el registro de los recursos. Además, tienen soporte inmediato para servicios nuevos apenas salen al mercado y se integran de forma directa con las políticas de seguridad nativas de la plataforma.

4. ¿Qué métricas consideran esenciales para el monitoreo de soluciones en la nube?

Para no llenarse de alertas que nadie lee, conviene dividir el monitoreo entre lo que siente el usuario final (las señales de servicio) y el estado interno de los recursos:

- **Latencia (Tiempo de respuesta):** Cuánto tarda el backend y la base de datos en responder una petición. Es importante medir percentiles (como p95 y p99) y no solo el promedio, para detectar lentitudes que afectan a transacciones específicas.
- **Tasa de Errores (Error Rate):** Porcentaje de respuestas fallidas. Separar errores HTTP 5xx (fallos propios del servidor, timeouts o caídas de base de datos) de los errores 4xx (peticiones mal formadas o bloqueos de permisos/WAF).
- **Tráfico y Concurrencia:** Volumen de peticiones por segundo (RPS), sesiones activas y ancho de banda consumido en el balanceador o API Gateway. Sirve tanto para dimensionar el autoescalado como para detectar picos anómalos.
- **Saturación de Cómputo y Base de Datos:** Uso de CPU, memoria RAM, IOPS de disco y, muy importante en bases de datos gestionadas, el consumo de DTUs/vCores, el porcentaje de conexiones activas frente al límite permitido y los bloqueos de tablas (deadlocks). Muchas veces una aplicación se cae porque se agota el pool de conexiones a la base de datos antes de que el procesador llegue al 100%.

- **Métricas de Seguridad y Costos (FinOps):** Intentos de inicio de sesión fallidos, reglas activadas en el WAF y alertas de desviación de presupuesto para detectar a tiempo si algún recurso escaló de forma descontrolada.

5. ¿Qué es Docker y cuáles son sus componentes principales?

Docker es una plataforma de contenedorización que permite empaquetar una aplicación junto con todas sus librerías, dependencias y variables de entorno en una unidad ligera y aislada. A diferencia de una máquina virtual tradicional —que necesita levantar un sistema operativo invitado completo sobre un hipervisor—, los contenedores de Docker comparten el mismo kernel del sistema operativo host y usan características nativas de Linux (namespaces para aislar procesos/red y cgroups para limitar CPU y memoria). Por eso arrancan en segundos y consumen muchos menos recursos.

Componentes principales de Docker:

- **Docker Engine (Motor Cliente-Servidor):** Está formado por el Docker Daemon (dockerd), que es el servicio en segundo plano encargado de crear y gestionar los contenedores; una API REST de comunicación; y el Docker CLI (el comando docker que usamos en la terminal).
- **Dockerfile:** Es el archivo de texto con las instrucciones paso a paso (imagen base, copia de código, instalación de paquetes, puerto expuesto y comando de arranque) para construir una imagen.
- **Imágenes (Images):** Son plantillas de solo lectura e inmutables que se construyen a partir del Dockerfile mediante un sistema de capas superpuestas. Si cambias una línea de código, solo se reconstruye esa capa, lo que agiliza mucho los pipelines de CI/CD.
- **Contenedores (Containers):** Son las instancias en ejecución de una imagen. A las capas de solo lectura de la imagen se les agrega una capa superior temporal de lectura/escritura mientras el contenedor está vivo.
- **Docker Registry (Registro de imágenes):** Es el repositorio donde se almacenan y distribuyen las imágenes construidas. En entornos empresariales no se deja el código en Docker Hub público, sino en registros privados como Azure Container Registry (ACR) o Amazon ECR, integrados con escaneo de vulnerabilidades.

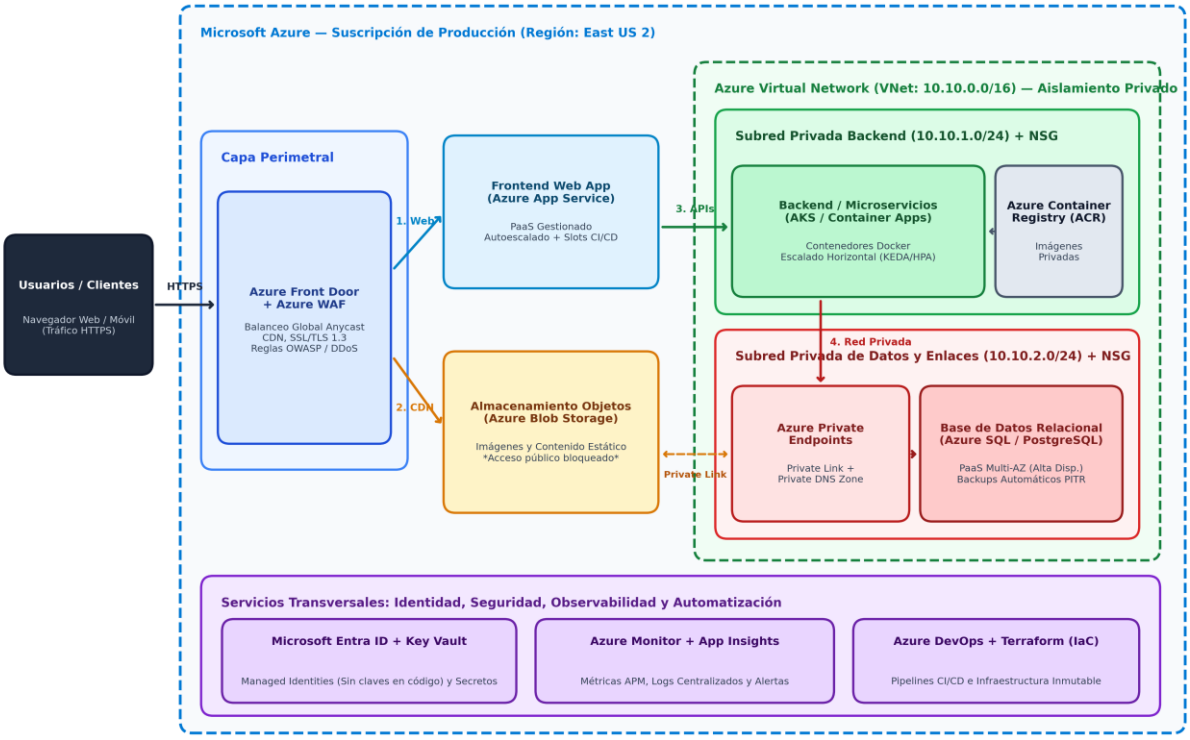
Parte II: Caso Práctico — Diseño de Arquitectura Cloud-Native

1. Selección del Proveedor de Nube: Microsoft Azure

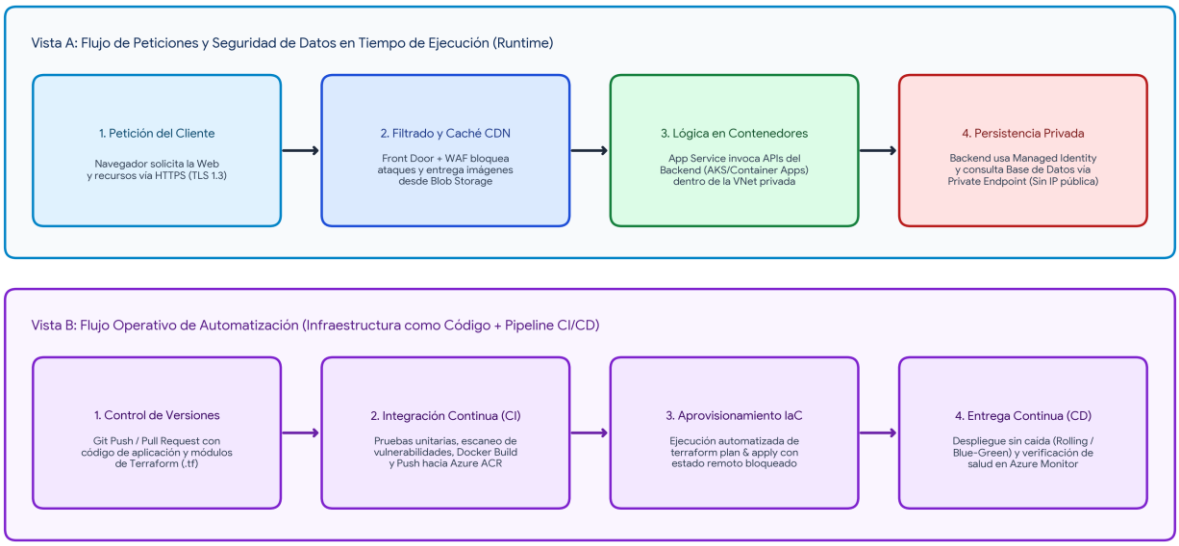
Para este diseño se ha seleccionado Microsoft Azure y se mantendrá todos los componentes dentro de su ecosistema. La elección se basa en la madurez de sus servicios PaaS (que reducen la carga operativa de administrar servidores base), sus capacidades nativas de red privada (Private Link) para aislar bases de datos y almacenamiento sin exponerlos a internet, y su integración directa con herramientas de gobierno, identidad (Microsoft Entra ID) y automatización con Azure DevOps / Terraform.

2. Diagrama de Arquitectura

2.1. Diagrama Principal: Topología de Componentes y Aislamiento de Red (VNet)



2.2. Diagrama Complementario: Flujo de Peticiones (Runtime) y Despliegue Automatizado (IaC + CI/CD)



Flujo general de la arquitectura (de extremo a extremo):

1) El usuario accede desde su navegador pasando primero por Azure Front Door (CDN + WAF).

2) El contenido estático e imágenes se sirven con baja latencia desde Azure Blob Storage a través del CDN, y la aplicación web principal se entrega desde el servicio de Frontend.

3) Las llamadas de negocio del Frontend viajan hacia el Backend alojado en contenedores dentro de una red virtual privada (VNet).

4) El Backend procesa la lógica y consulta la base de datos gestionada (Azure SQL Database / PostgreSQL) y el almacenamiento de objetos a través de Private Endpoints, sin que los datos salgan jamás a internet público.

3. Componentes Seleccionados y Justificación de Diseño

Capa Requerida	Servicio en Microsoft Azure	Rol en la Arquitectura
Seguridad Perimetral y Entrega	Azure Front Door + Azure WAF	Punto único de entrada HTTPS, protección contra ataques (OWASP Top 10 / DDoS) y caché de contenido estático.
Frontend (Aplicación Web)	Azure App Service (o Azure Static Web Apps)	Hospedaje gestionado para la aplicación web de cara al cliente con certificados TLS y autoescalado.
Backend (Servicios / APIs)	Azure Container Apps / Azure Kubernetes Service (AKS)	Ejecución de servicios en contenedores desacoplados, con

		escalado automático según demanda e integración a la VNet.
Base de Datos	Azure Database for PostgreSQL / Azure SQL Database	Motor relacional PaaS con alta disponibilidad (Zone-Redundant), backups automáticos y acceso exclusivo por red privada.
Almacenamiento de Objetos	Azure Blob Storage	Repositorio para imágenes y archivos estáticos con políticas de ciclo de vida y bloqueo de acceso público directo.
Seguridad, Secretos y Monitoreo	Entra ID, Key Vault y Azure Monitor (App Insights)	Autenticación sin contraseñas (Managed Identity), resguardo de llaves y trazabilidad completa de métricas y logs.

Detalle y justificación por componente:

- **Entrada y Seguridad Perimetral (Azure Front Door + WAF):** En lugar de exponer directamente el frontend o el almacenamiento, ponemos Azure Front Door al frente. Actúa como balanceador global, termina las conexiones SSL/TLS y filtra ataques comunes (inyección SQL, XSS, bots) mediante el Web Application Firewall (WAF) antes de que el tráfico toque nuestra infraestructura.
- **Frontend (Azure App Service):** Los clientes navegan en la aplicación web alojada en App Service. Se elige un servicio PaaS porque nos ahorra parchar sistemas operativos y nos da escalado horizontal automático en horas pico, además de permitir despliegues sin caída (Deployment Slots para pruebas Blue/Green) desde el pipeline de CI/CD.
- **Almacenamiento de Objetos (Azure Blob Storage):** Para no saturar el cómputo del frontend ni del backend sirviendo fotos, documentos o recursos estáticos (JS, CSS, imágenes de catálogo), todo ese contenido se guarda en Blob Storage. Por seguridad, la cuenta de almacenamiento tiene el acceso público deshabilitado; los archivos se entregan únicamente a través de Azure Front Door (usando acceso privado) o mediante tokens temporales firmados (SAS Tokens) generados por el backend cuando un usuario sube o consulta un archivo específico.
- **Backend en Contenedores (Azure Kubernetes Service - AKS / Azure Container Apps):** Los servicios de negocio que procesan las peticiones del frontend y hablan con la base de datos corren en contenedores Docker obtenidos desde un registro privado (Azure Container Registry). Esto permite escalar cada servicio de forma independiente según el consumo de CPU, memoria o cola de peticiones. El backend vive dentro de una subred privada en una Virtual Network (VNet) y solo acepta tráfico proveniente de la capa de entrada/API.

- **Base de Datos (Azure SQL Database / Azure Database for PostgreSQL Flexible Server):** Se utiliza un motor de base de datos totalmente administrado en modalidad Multi-AZ (redundancia de zona) para garantizar continuidad si falla un centro de datos físico dentro de la región. Cuenta con copias de seguridad automáticas con restauración a un punto en el tiempo (PITR). La base de datos no tiene IP pública; el backend se conecta a ella mediante un Private Endpoint dentro de la VNet y utilizando Managed Identity para evitar guardar usuarios y contraseñas en archivos de configuración.
- **Operación, Secretos y Observabilidad (Key Vault + Azure Monitor):** Cualquier certificado o clave externa se almacena en Azure Key Vault. A nivel de monitoreo, todos los componentes envían sus métricas y logs a un Log Analytics Workspace centralizado junto con Application Insights en el backend, lo que permite rastrear en tiempo real la latencia de las consultas a la base de datos, errores HTTP y el estado de salud de los contenedores.